

Compte Rendu de Travaux Pratiques

Base de données relationnelle PostgreSQL
Plateforme logistique — Gestion clients / commandes / articles

Étape	Compétences	Score
1 — Création tables	DDL, CREATE TABLE, SERIAL	3 / 3
2 — Table liaison	N:N, clé composite, CHECK	3 / 3
3 — Insertions	DML, INSERT INTO	3 / 3
4 — Requêtes simples	SELECT, WHERE, ORDER BY	3 / 3
5 — Jointures	INNER JOIN multi-tables	3 / 3
6 — Agrégats	GROUP BY, HAVING	3 / 3
7 — Sous-requêtes	MAX(), RANK() OVER	2 / 2
Total		20 / 20

Introduction

Ce TP porte sur la création et la manipulation d'une base de données relationnelle PostgreSQL simulant une plateforme logistique. L'objectif est de mettre en pratique les compétences fondamentales en SQL : création de tables, définition de contraintes, insertion de données, requêtes de sélection, jointures, agrégations et sous-requêtes.

Schéma relationnel

```
client ( id_client, nom, ville )

article ( id_article, designation, prix )

commande ( id_commande, date_commande, mode_paiement, id_client )

contient ( id_commande, id_article, quantite ) -- liaison N:N
```

Un client peut avoir plusieurs commandes. Une commande peut contenir plusieurs articles. La table 'contient' réalise la relation N:N entre commande et article.

Étape 1 — Création des tables principales

3/3

Compétences : DDL, CREATE TABLE, SERIAL, NOT NULL, PRIMARY KEY, types de données.

Avant toute manipulation de données, il faut définir la structure de la base. CREATE TABLE définit les colonnes, types et contraintes. Le mot-clé SERIAL crée automatiquement une séquence d'entiers auto-incrémentés.

1.1 — Table client

```
CREATE TABLE client (

  id_client SERIAL PRIMARY KEY,

  nom VARCHAR(50) NOT NULL,

  ville VARCHAR(50) NOT NULL

);
```

SERIAL PRIMARY KEY : PostgreSQL crée la séquence client_id_client_seq. PRIMARY KEY garantit l'unicité et interdit NULL.

1.2 — Table article

```
CREATE TABLE article (

  id_article SERIAL PRIMARY KEY,

  designation VARCHAR(100) NOT NULL,

  prix DECIMAL(10, 2) NOT NULL

);
```

DECIMAL(10,2) : type numérique exact, idéal pour les prix (évite les erreurs d'arrondi des types FLOAT/DOUBLE).

1.3 — Table commande

```
CREATE TABLE commande (
```

```
id_commande SERIAL PRIMARY KEY,  
  
date_commande DATE NOT NULL,  
  
mode_paiement VARCHAR(20) NOT NULL,  
  
id_client INT NOT NULL,  
  
FOREIGN KEY (id_client) REFERENCES client(id_client) ON DELETE CASCADE  
  
);
```

ON DELETE CASCADE : suppression automatique des commandes liées quand un client est supprimé.

Résultat — \d dans psql

```
tp_users=> \d  
  
public | article | table | <mon_nom>  
  
public | article_id_article_seq | sequence | <mon_nom>  
  
public | client | table | <mon_nom>  
  
public | client_id_client_seq | sequence | <mon_nom>  
  
public | commande | table | <mon_nom>  
  
public | commande_id_commande_seq | sequence | <mon_nom>  
  
(6 rows)
```

6 relations : 3 tables + 3 séquences (une par SERIAL).

Étape 2 — Ajout des relations — table de liaison contient

3/3

Compétences : modélisation N:N, clé primaire composite, double clé étrangère, contrainte CHECK.

Une commande peut contenir plusieurs articles, et un même article peut figurer dans plusieurs commandes. Cette relation N:N nécessite une table intermédiaire.

Version initiale (incorrecte — clé primaire trop restrictive)

```
CREATE TABLE contient (  
    id_commande INT NOT NULL,  
    id_article INT NOT NULL,  
    PRIMARY KEY (id_commande), -- ERREUR : 1 seul article par commande !  
    FOREIGN KEY (id_commande) REFERENCES commande(id_commande) ON DELETE CASCADE,  
    FOREIGN KEY (id_article) REFERENCES article(id_article) ON DELETE CASCADE  
);
```

Problème : avec PRIMARY KEY sur id_commande seul, une commande ne peut contenir qu'un article. La clé primaire doit être composite.

Version corrigée (clé primaire composite + quantité)

```
CREATE TABLE contient (  
    id_commande INT NOT NULL,  
    id_article INT NOT NULL,  
    quantite INT NOT NULL CHECK (quantite > 0),  
    PRIMARY KEY (id_commande, id_article),  
    FOREIGN KEY (id_commande) REFERENCES commande(id_commande) ON DELETE CASCADE,  
    FOREIGN KEY (id_article) REFERENCES article(id_article) ON DELETE CASCADE  
);
```

CHECK (quantite > 0) : contrainte de domaine enforcee par le SGBD. Toute insertion avec quantite = 0 ou négatif sera rejetée.

Étape 3 — Insertion des données

3/3

Compétences : DML, INSERT INTO ... VALUES (multi-lignes), respect de l'ordre d'insertion (FK).

3.1 — 4 clients

```
INSERT INTO client (id_client, nom, ville) VALUES  
  
(1, 'Massin', 'Haubourdin'),  
  
(2, 'Lemaitre', 'Lillier'),  
  
(3, 'Panier', 'Arras'),
```

```
(4, 'Sauvage', 'Lens');
```

```
id_client | nom | ville
-----+-----+-----
1 | Massin | Haubourdin
2 | Lemaitre | Lillier
3 | Panier | Arras
4 | Sauvage | Lens
(4 rows)
```

3.2 — 5 articles

```
INSERT INTO article (id_article, designation, prix) VALUES
(1, 'banane', 1.00),
(2, 't-shirt', 11.00),
(3, 'Funko Pop', 25.00),
(4, 'Haribos', 4.00),
(5, 'Clio 3', 5799.99);
```

```
id_article | designation | prix
-----+-----+-----
1 | banane | 1.00
2 | t-shirt | 11.00
3 | Funko Pop | 25.00
5 | Clio 3 | 5799.99
4 | Haribos | 4.00
(5 rows)
```

L'ordre d'affichage (5 avant 4) est dû à l'ordre physique d'insertion. Sans ORDER BY, PostgreSQL ne garantit pas d'ordre particulier.

3.3 — 6 commandes

```
INSERT INTO commande (date_commande, mode_paiement, id_client) VALUES
('2026-01-01', 'CB', 1),
('2026-02-23', 'CB', 4),
('2026-03-03', 'Espece', 3),
('2026-03-03', 'CB', 2),
('2026-06-08', 'Espece', 1),
('2026-12-14', 'CB', 3);
```

```

id_commande | date_commande | mode_paiement | id_client
-----+-----+-----+-----
1 | 2026-01-01 | CB | 1
2 | 2026-02-23 | CB | 4
3 | 2026-03-03 | Espece | 3
4 | 2026-03-03 | CB | 2
5 | 2026-06-08 | Espece | 1
6 | 2026-12-14 | CB | 3

(6 rows)

```

3.4 — 8 lignes dans contient

```

INSERT INTO contient (id_commande, id_article, quantite) VALUES

(1, 1, 3), -- commande 1 : 3 bananes

(1, 2, 1), -- commande 1 : 1 t-shirt

(2, 3, 2), -- commande 2 : 2 Funko Pop

(3, 1, 5), -- commande 3 : 5 bananes

(3, 4, 2), -- commande 3 : 2 Haribos

(4, 2, 3), -- commande 4 : 3 t-shirts

(5, 5, 1), -- commande 5 : 1 Clio 3

(6, 3, 1); -- commande 6 : 1 Funko Pop

```

L'ordre d'insertion respecte les FK : clients avant commandes, commandes et articles avant contient.

Compétences : *SELECT, WHERE, ORDER BY, filtres par valeur.*

4.1 — Articles triés par prix croissant

```
SELECT * FROM article ORDER BY prix;
```

```
id_article | designation | prix
-----+-----+-----
1 | banane | 1.00
4 | Haribos | 4.00
2 | t-shirt | 11.00
3 | Funko Pop | 25.00
5 | Clio 3 | 5799.99
(5 rows)
```

ORDER BY prix sans ASC/DESC trie par défaut en croissant. Pour décroissant : ORDER BY prix DESC.

4.2 — Clients habitant Lillier

```
SELECT * FROM client WHERE ville = 'Lillier';
```

```
id_client | nom | ville
-----+-----+-----
2 | Lemaitre | Lillier
(1 row)
```

PostgreSQL est sensible à la casse : 'lillier' != 'Lillier'. Pour insensible : WHERE ville ILIKE 'lillier'.

4.3 — Commandes avec mode_paiement = CB

```
SELECT * FROM commande WHERE mode_paiement = 'CB';
```

```
id_commande | date_commande | mode_paiement | id_client
-----+-----+-----+-----
1 | 2026-01-01 | CB | 1
2 | 2026-02-23 | CB | 4
4 | 2026-03-03 | CB | 2
6 | 2026-12-14 | CB | 3
(4 rows)
```

Sur 6 commandes totales : 4 par CB et 2 en espèces. On peut combiner avec AND/OR pour des filtres multiples.

Compétences : INNER JOIN, liaison de tables via clés étrangères, jointures multi-tables.

Les jointures combinent les lignes de deux tables ou plus quand la condition de jointure est satisfaite. INNER JOIN ne retourne que les lignes ayant une correspondance dans les deux tables.

5.1 — Nom du client + date de commande

```
SELECT client.nom, commande.date_commande  
  
FROM client  
  
INNER JOIN commande ON client.id_client = commande.id_client;
```

```
nom | date_commande  
-----+-----  
Massin | 2026-01-01  
Sauvage | 2026-02-23  
Panier | 2026-03-03  
Lemaitre | 2026-03-03  
Massin | 2026-06-08  
Panier | 2026-12-14  
  
(6 rows)
```

Massin et Panier apparaissent deux fois car ils ont passé 2 commandes. INNER JOIN crée autant de lignes qu'il y a de correspondances.

5.2 — Désignation d'article + quantité commandée

```
SELECT article.designation, contient.quantite  
  
FROM article  
  
INNER JOIN contient ON article.id_article = contient.id_article  
  
ORDER BY article.designation;
```

```
designation | quantite  
-----+-----  
Clio 3 | 1  
Funko Pop | 2  
Funko Pop | 1  
Haribos | 2  
banane | 3  
banane | 5  
t-shirt | 1  
t-shirt | 3  
  
(8 rows)
```

Les 8 lignes retournées correspondent exactement aux 8 lignes de la table contient.

Étape 6 — Agrégats (GROUP BY / HAVING)

3/3

Compétences : GROUP BY, SUM(), HAVING pour filtrer les groupes.

GROUP BY regroupe les lignes ayant la même valeur dans une colonne, puis applique une fonction d'agrégation (SUM, COUNT, AVG...) sur chaque groupe. HAVING filtre les groupes résultants.

6.1 — Total des quantités commandées par article

```
SELECT article.designation, SUM(contient.quantite) AS total_quantite
FROM article
INNER JOIN contient ON article.id_article = contient.id_article
GROUP BY article.designation
ORDER BY total_quantite DESC;
```

```
designation | total_quantite
-----+-----
banane | 8
t-shirt | 4
Funko Pop | 3
Haribos | 2
Clio 3 | 1
```

La Clio 3 est l'article le moins commandé (1 unité), la banane le plus commandé (8 unités).

6.2 — Articles avec quantité totale supérieure à 5

```
SELECT article.designation, SUM(contient.quantite) AS total_quantite
FROM article
INNER JOIN contient ON article.id_article = contient.id_article
GROUP BY article.designation
HAVING SUM(contient.quantite) > 5;
```

```
designation | total_quantite
-----+-----
banane | 8
(1 row)
```

HAVING vs WHERE : on ne peut pas écrire WHERE SUM(...) > 5 car WHERE s'applique avant le calcul des agrégats. HAVING s'applique après GROUP BY.

Compétences : sous-requête scalaire avec MAX(), fonction analytique RANK() OVER.

7.1 — Prix maximum (agrégat simple)

```
SELECT MAX(prix) AS prix_max FROM article;
```

```
prix_max
-----
5799.99
(1 row)
```

7.2 — Article complet le plus cher (sous-requête)

Pour récupérer tous les détails de l'article le plus cher, on utilise le résultat de MAX comme sous-requête scalaire :

```
SELECT * FROM article
WHERE prix = (SELECT MAX(prix) FROM article);
```

```
id_article | designation | prix
-----+-----+-----
5 | Clio 3 | 5799.99
(1 row)
```

La sous-requête s'exécute en premier et retourne 5799.99, utilisé comme valeur de comparaison dans le WHERE de la requête principale.

7.3 — Rang des articles selon le prix (fonction window)

Les fonctions window calculent une valeur pour chaque ligne en tenant compte d'un ensemble de lignes voisines, sans regrouper les résultats :

```
SELECT
    designation,
    prix,
    RANK() OVER (ORDER BY prix DESC) AS rang
FROM article;
```

```
designation | prix | rang
-----+-----+-----
Clio 3 | 5799.99 | 1
Funko Pop | 25.00 | 2
t-shirt | 11.00 | 3
Haribos | 4.00 | 4
banane | 1.00 | 5
(5 rows)
```

RANK() laisse des trous si ex-aequo (1,2,2,4...). *DENSE_RANK()* n'en laisse pas (1,2,2,3...). *ROW_NUMBER()* numérote séquentiellement.